

Semester	T.E. Semester VI – INFT
Subject	Distributed Computing
Laboratory Teacher	Prof. Dhanashree Tamhane
Laboratory	CC-03

Project Title	Game Library Management System
Experiment No	3
Experiment Title:	Lamport Clock Synchronization

Sr No.	Full Name	Roll Number
1	Sarthak Kulkarni	23101B0019
2	Harshad Patekar	23101B0026
3	Tanay Shinde	23101B0034
4	Yash Patil	23101B0060

Aim	To implement Lamport Logical Clock synchronization in a distributed application (gRPC-based Continuum project) and verify that events occurring across different services are assigned a consistent, causally correct logical ordering.
Theory	Distributed Systems and Clock Synchronization In a distributed system, multiple processes run concurrently on different machines. Physical clocks (wall-clock time) cannot be perfectly synchronized due to network latency, clock drift, and the absence of a global time authority. Logical clocks are therefore used to order events causally rather than physically. Lamport Logical Clock Proposed by Leslie Lamport in 1978, the Lamport Logical Clock defines a partial ordering of events in a distributed system. Each process maintains a counter (logical clock) that follows three rules: 1. Local Event: Before executing a local event, increment the clock - $Clock = Clock + 1$. 2. Sending a Message: Before sending a message, increment the clock and attach the timestamp to

the message - $\text{Clock} = \text{Clock} + 1$; $\text{Send}(\text{message}, \text{timestamp})$.
 3. Receiving a Message: Upon receiving a message with timestamp T , update the local clock as $\text{Clock} = \max(\text{Clock}, T) + 1$.

Causality and the Happened-Before Relation

- If event a happens before event b in the same process, then $C(a) < C(b)$.
- If a is sending a message and b is receiving that same message, then $C(a) < C(b)$.
- If $C(a) < C(b)$, then a happened before b (causally).
- If $C(a) == C(b)$, the events are concurrent (neither caused the other).

Properties

- Monotonicity: Timestamps always increase along causal paths.
- Consistency: All processes agree on the order of causally related events.
- Concurrency: Concurrent events may have the same or incomparable timestamps.

Application in Continuum Project

A LamportClock module was integrated into both the Order/Client service and the Inventory/Server service of the Continuum – Game Discovery & Library Tracker project. The client ticks its clock before sending a GetGameStatus request and attaches the timestamp; the server updates its clock on receipt, ticks again before responding, and the client updates once more on receiving the response - demonstrating consistent causal ordering across two distributed services using gRPC.

Components	Component	Description
	LamportClock	Class implementing tick(), update(), and get_time()
	continuum.proto	Protobuf definition with lamport_timestamp field
	client.py	gRPC client integrating the Lamport clock
	server.py	gRPC server integrating the Lamport clock
	server_lamport.log	Server-side event log
	client_lamport.log	Client-side event log
Procedure	Step 1 Used the existing gRPC-based Continuum project (grpc/ directory) with the client (client.py) and server (server.py) components already in place. Step 2 Created grpc/lamport_clock.py with a LamportClock class implementing tick(), update(), and get_time(). Step 3 Updated grpc/proto/continuum.proto to add a lamport_timestamp field to both GameRequest and GameResponse. Step 4 Regenerated the gRPC files: <code>python -m grpc_tools.protoc -I./grpc/proto --python_out=./grpc --grpc_python_out=./grpc ./grpc/proto/continuum.proto</code>	

Step 5

Modified `grpc/client.py` to initialize a `LamportClock`, call `tick()` before sending the request, attach the timestamp to the `GameRequest`, and call `update()` upon receiving the response.

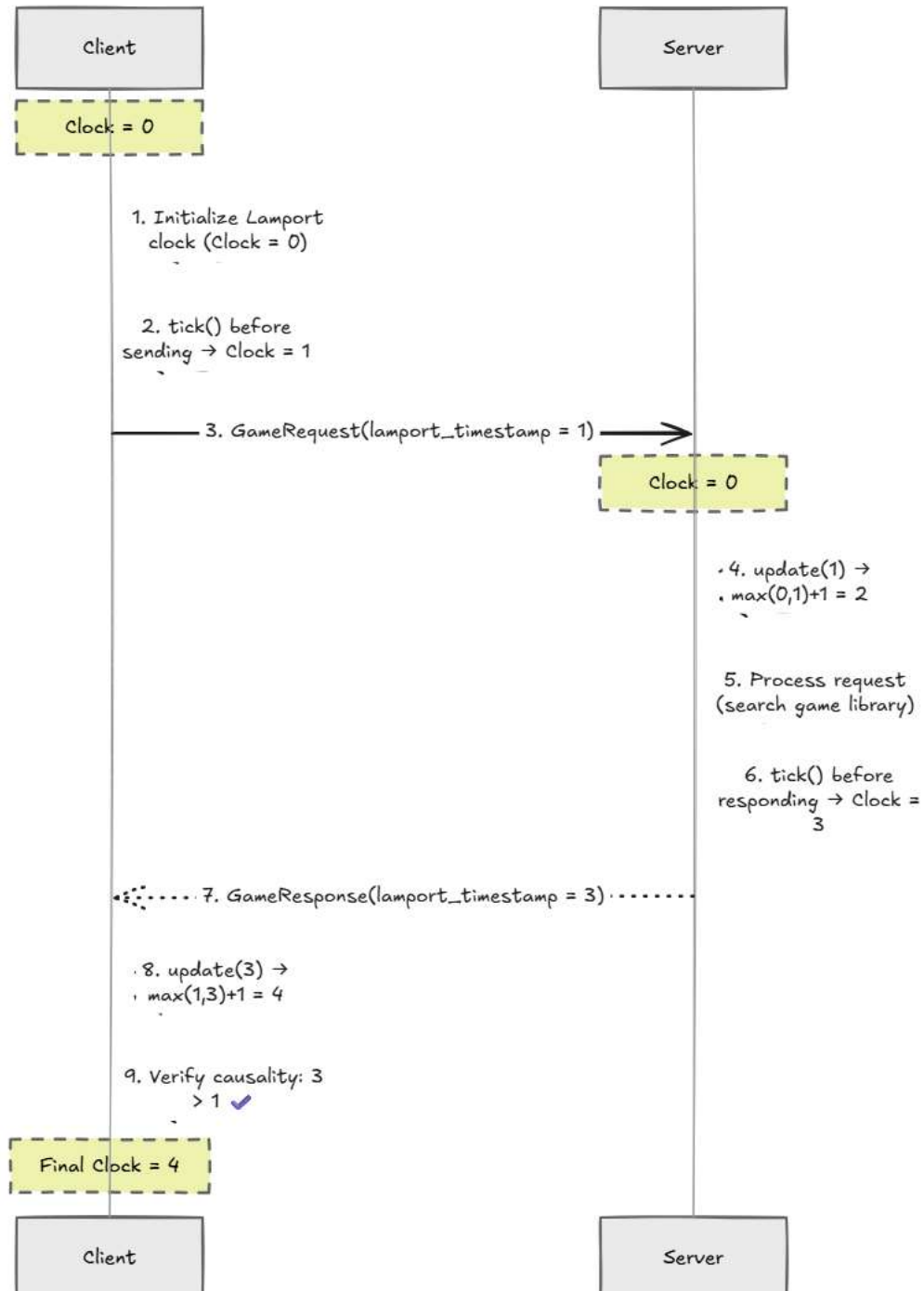
Step 6

Modified `grpc/server.py` to initialize a `LamportClock`, call `update()` upon receiving the request, call `tick()` before sending the response, and attach the timestamp to the `GameResponse`.

Step 7

Started the server (`python grpc/server.py`) and the client (`python grpc/client.py`) in separate terminals, observed the Lamport timestamp progression, and verified causality and clock-rule compliance.

Algorithm



● ● ● lamport_clock.py

Python

```
1 class LamportClock:
2     def __init__(self, node_id="unknown"):
3         self.counter = 0
4         self.node_id = node_id
5
6     def tick(self):
7         self.counter += 1
8         return self.counter
9
10    def update(self, received_timestamp):
11        self.counter = max(self.counter, received_timestamp) + 1
12        return self.counter
13
14    def get_time(self):
15        return self.counter
16
```

● ● ● continuum.proto

Java

```
1 syntax = "proto3";
2
3 package continuum;
4
5 service GameService {
6     rpc GetGameStatus(GameRequest) returns (GameResponse);
7 }
8
9 message GameRequest {
10     string game_name = 1;
11     int64 lamport_timestamp = 2;
12 }
13
14 message GameResponse {
15     string status = 1;
16     int64 lamport_timestamp = 2;
17 }
```

Output

Server Side

```
=====
[SERVER] Starting gRPC Server...
=====
[SERVER] gRPC Server Started Successfully
[SERVER] Listening on Port : 50051
[SERVER] Waiting for Client Requests...
[SERVER] Logs are being written to: server_lamport.log
=====

=====
[SERVER] Request #1 Received
[SERVER] Requested Game : Elden Ring
[SERVER] Client Lamport Timestamp : 1
[SERVER] Server Lamport Clock (before update) : 0
[SERVER] Server Lamport Clock (after update) : 2
[SERVER] Step 2 : Searching Game Library...
[SERVER] Game Found!
[SERVER] Step 3 : Preparing Response...
[SERVER] Response Lamport Timestamp : 3
[SERVER] Response Ready - Sending to Client...
=====
```

Client Side

```
=====
[CLIENT] Continuum Game Client Started
=====
[CLIENT] Step 1 : Connecting to gRPC Server...
[CLIENT] Connected Successfully
[CLIENT] Step 2 : Initializing Lamport Clock...
[CLIENT] Step 3 : Creating Request
[CLIENT] Game Requested : Elden Ring
[CLIENT] Client Lamport Timestamp (send) : 1
[CLIENT] Step 4 : Sending Request to Server...
[CLIENT] Step 5 : Response Received
-----
[CLIENT] Response Status : Elden Ring is currently in your Playing Library.
[CLIENT] Server Lamport Timestamp : 3
[CLIENT] Client Lamport Clock (before update) : 1
[CLIENT] Client Lamport Clock (after update) : 4
-----
[CLIENT] VERIFICATION
-----
[CLIENT] TP (True Positive) : Server timestamp (3) > Client send timestamp (1)
[CLIENT] Causality preserved: Message was received after it was sent
[CLIENT] Clock update correct: 1 -> 4
[CLIENT] Final Client Lamport Clock : 4
-----
[CLIENT] Experiment Completed Successfully
[CLIENT] Logs are being written to: client_lamport.log
=====
```

Timestamp Progression	<table><tr><th>Step</th><th>Component</th><th>Event</th><th>Lamport Timestamp</th></tr><tr><td>1</td><td>Client</td><td>Local event (initialize clock)</td><td>0</td></tr><tr><td>2</td><td>Client</td><td>tick() before send</td><td>1</td></tr><tr><td>3</td><td>Server</td><td>update(1) on receive</td><td>2</td></tr><tr><td>4</td><td>Server</td><td>tick() before response</td><td>3</td></tr><tr><td>5</td><td>Client</td><td>update(3) on receive</td><td>4</td></tr></table>	Step	Component	Event	Lamport Timestamp	1	Client	Local event (initialize clock)	0	2	Client	tick() before send	1	3	Server	update(1) on receive	2	4	Server	tick() before response	3	5	Client	update(3) on receive	4
Step	Component	Event	Lamport Timestamp																						
1	Client	Local event (initialize clock)	0																						
2	Client	tick() before send	1																						
3	Server	update(1) on receive	2																						
4	Server	tick() before response	3																						
5	Client	update(3) on receive	4																						
Verification & Analysis	1. Rule Verification - Send Rule: Client called tick() → 0 + 1 = 1. ✓ - Receive Rule: Server called update(1) → max(0, 1) + 1 = 2. ✓ - Response Rule: Server called tick() → 2 + 1 = 3. ✓ - Client Update Rule: Client called update(3) → max(1, 3) + 1 = 4. ✓ 2. Causality Verification - TP (True Positive): Server timestamp (3) > Client send timestamp (1) — causality preserved; the message was received after it was sent. 3. FN (False Negative): None detected — no causality violations. - Clock Monotonicity: Both client and server clocks strictly increased along the causal path.																								
Result	The Lamport Logical Clock synchronization was successfully implemented and verified in the Continuum gRPC application. Client and server timestamps increased monotonically along the causal path, and the receive rule Clock = max(Clock, T) + 1 was confirmed at every step, demonstrating consistent causal ordering across distributed services.																								
Conclusion	The experiment demonstrates that (1) each process maintains its own logical clock, (2) a sender increments its clock and attaches the timestamp before sending a message, (3) a receiver updates its clock using max(local, received) + 1, and (4) causally related events receive monotonically increasing timestamps. The output confirms the Lamport clock rules were correctly followed, and the system preserves causality without requiring physical clock synchronization.																								